

TECNOLOGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

Agosto - Diciembre 2025

CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:

Patrones de diseño

TÍTULO ACTIVIDAD:

Examen unidad 2

UNIDAD A EVALUAR:

Unidad 2

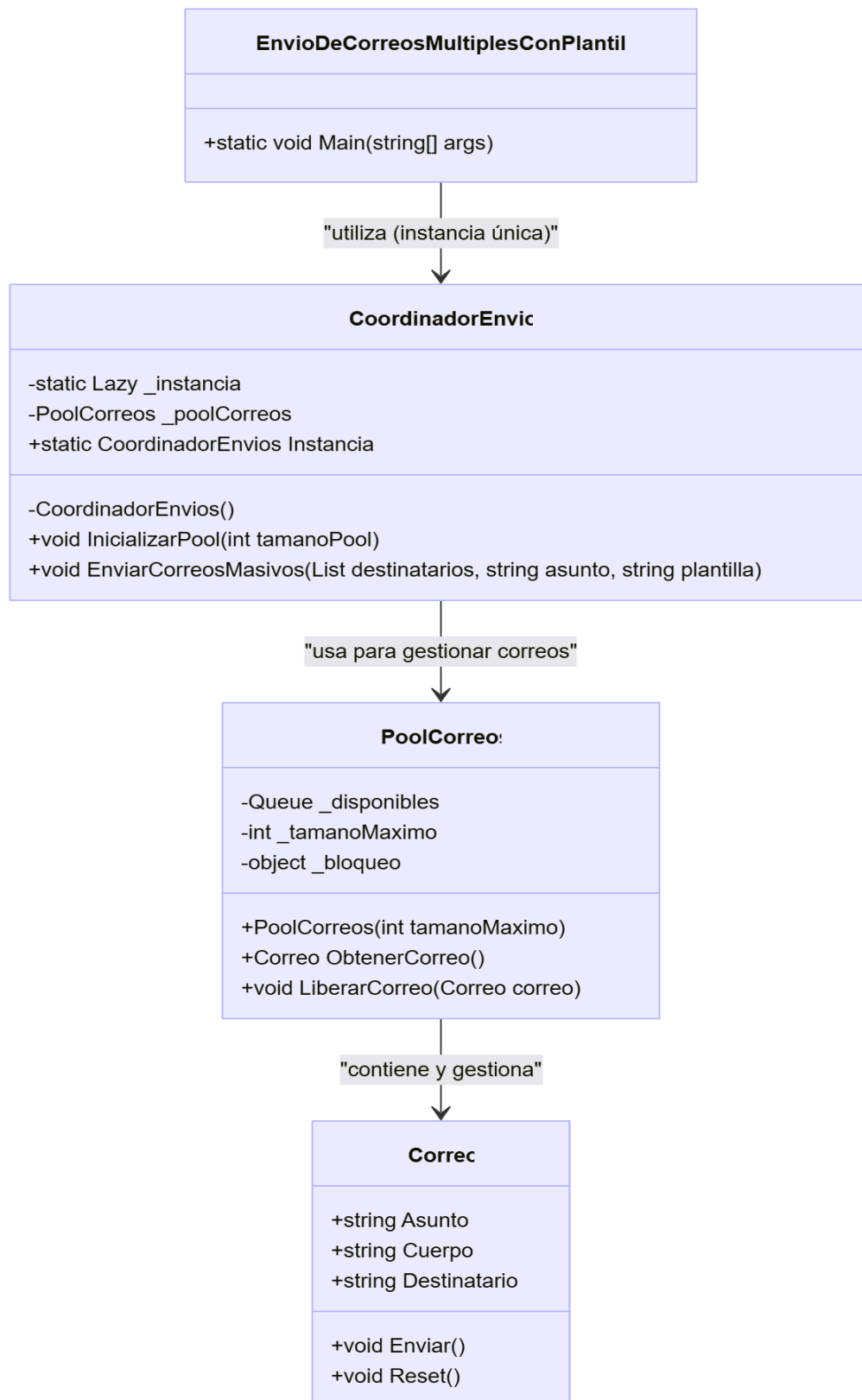
NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Ceballos Resendiz Veronica Sarahi 20211759

NOMBRE DEL MAESTRO (A):

Maribel Guerrero Luis

Diagrama UML



Código

```
using System;
```

```
using System.Collections.Generic;
```

```
namespace EmailSystem
```

```
{
```

```
    //Esta clase representa un correo reutilizable dentro del pool
```

```
    public class Correo
```

```
    {
```

```
        //Propiedad para el asunto del correo
```

```
        public string Asunto { get; set; }
```

```
        //Propiedad para el cuerpo del correo
```

```
        public string Cuerpo { get; set; }
```

```
        //Propiedad para el destinatario del correo
```

```
        public string Destinatario { get; set; }
```

```
        //Método que "envía" el correo (aquí simulado con Console.WriteLine)
```

```
        public void Enviar()
```

```
        {
```

```
            //Muestra el destinatario
```

```
            Console.WriteLine($"Enviando correo a: {Destinatario}");
```

```
            //Muestra el asunto
```

```
            Console.WriteLine($"Asunto: {Asunto}");
```

```
            //Muestra el cuerpo
```

```
            Console.WriteLine($"Cuerpo: buenas tardes profesora maribel \n{Cuerpo}");
```

```
        Console.WriteLine("-----");  
    }
```

//Reinicia el estado del objeto para que pueda reutilizarse sin restos de datos previos

```
    public void Reset()  
    {  
        //Borra asunto  
        Asunto = string.Empty;  
  
        //cuerpo  
        Cuerpo = string.Empty;  
  
        //Borra destinatario  
        Destinatario = string.Empty;  
    }  
}
```

//se implementa del patrón Object Pool para objetos Correo

public class PoolCorreos

```
{  
    //cola de los obeitos que estan disponibles  
    private readonly Queue<Correo> _disponibles = new Queue<Correo>();  
  
    //maximo numero de objetos en el pool  
    private readonly int _tamanoMaximo;  
  
    //objeto para sincronizacion o lock  
    private readonly object _bloqueo = new object();  
  
    //Constructor que precrea 'tamanoMaximo' objetos Correo y los encola
```

//este fragmento de código representa al constructor de la clase poolcorreos y aquí es donde se encarga de

//inicializar el pool o el conjunto de objetos disponibles

```
public PoolCorreos(int tamanoMaximo)
```

```
{
```

```
    //aquí se guarda el tamaño máximo
```

```
    _tamanoMaximo = tamanoMaximo;
```

//aquí se usa un bucle para la creación de los objetos iniciales que es la cantidad de los correos

```
    for (int i = 0; i < tamanoMaximo; i++)
```

```
        //Aquí se agrega cada nuevo correo a la cola que estén disponibles
```

```
        _disponibles.Enqueue(new Correo());
```

```
}
```

//devuelve un objeto Correo disponible del pool; lanza excepción si no hay disponibles

```
public Correo ObtenerCorreo()
```

```
{
```

```
    lock (_bloqueo) //asegura que solo un hilo a la vez acceda a la cola
```

```
{
```

```
    if (_disponibles.Count > 0) //si hay al menos un correo disponible
```

```
{
```

return _disponibles.Dequeue(); //saca y retorna el primer correo disponible

```
}
```

```
else
```

```
{
```

//aquí se lanza una excepción que indica cuando ya no hay objetos disponibles: decisión de diseño a lanzar excepción

```
        throw new InvalidOperationException("No hay correos disponibles en el  
pool.");
```

```
    }
```

```
}
```

```
}
```

```
//devuelve o libera un objeto Correo al pool para su reutilización
```

```
public void LiberarCorreo(Correo correo)
```

```
{
```

```
    lock (_bloqueo) //bloquea para evitar condiciones de carrera al encolar
```

```
{
```

```
    correo.Reset(); //limpia el estado del correo antes de reusar
```

```
    if (_disponibles.Count < _tamanoMaximo) //asegura que no sobrepase el  
tamaño máximo
```

```
{
```

```
    _disponibles.Enqueue(correo); //vuelve a encolar el correo para su  
reutilización
```

```
}
```

```
    //si por alguna razón la cola ya está llena, simplemente se descarta el  
objeto
```

```
}
```

```
}
```

```
}
```

```
//implementación del patrón Singleton: coordinador único de envíos
```

```
public sealed class CoordinadorEnvios
```

```
{
```

```
    //lazy<T> garantiza inicialización perezosa y thread-safe de la instancia única
```

```

private static readonly Lazy<CoordinadorEnvios> _instancia =
    new Lazy<CoordinadorEnvios>(() => new CoordinadorEnvios());

private PoolCorreos _poolCorreos; //referencia al pool de correos administrado
por el singleton

//constructor privado impide que otros creen instancias de CoordinadorEnvios
private CoordinadorEnvios() { }

//propiedad pública para acceder a la instancia única del singleton
public static CoordinadorEnvios Instancia => _instancia.Value;

//inicializa el pool solo si no ha sido inicializado anteriormente (idempotente)
public void InicializarPool(int tamanoPool)
{
    if (_poolCorreos == null) //si aún no existe el pool
    {
        _poolCorreos = new PoolCorreos(tamanoPool); //crea e inicializa el pool
        Console.WriteLine($"Pool de correos inicializado con {tamanoPool}
objetos.");
    }
    else
    {
        //si ya existía, solo notificamos; evitamos re-inicializar y perder objetos en
uso
        Console.WriteLine("El pool ya fue inicializado previamente.");
    }
}

```

```
//metodo para enviar correos masivos usando objetos del pool

public void EnviarCorreosMasivos(List<string> destinatarios, string asunto,
string plantilla)
{
    if (_poolCorreos == null) //validación: el pool debe estar inicializado
    {
        throw new InvalidOperationException("El pool de correos no ha sido
inicializado.");
    }

    //para cada destinatario en la lista
    foreach (var destinatario in destinatarios)
    {
        try
        {
            var correo = _poolCorreos.ObtenerCorreo(); //toma un objeto Correo
libre del pool

            //configura el correo con los datos deseados
            correo.Asunto = asunto;

            correo.Cuerpo = plantilla.Replace("{nombre}", destinatario); //reemplaza
marcador en plantilla

            correo.Destinatario = destinatario;

            correo.Enviar(); //llama al método que simula el envío

            _poolCorreos.LiberarCorreo(correo); //devuelve el objeto al pool para su
reutilización
```

```

    }

    catch (Exception ex)

    {
        //manejo básico de errores: informar y continuar con el siguiente
destinatario

        Console.WriteLine($"Error enviando correo a {destinatario}:
{ex.Message}");

    }

}

}

}

}

// Programa de prueba / punto de entrada

class Program

{

    static void Main(string[] args)

    {

        // Obtención de la instancia singleton (ambas referencias apuntarán a la
misma instancia)

        var coordinador1 = CoordinadorEnvios.Instancia;

        var coordinador2 = CoordinadorEnvios.Instancia;


        // Verificación de que ambas referencias son exactamente el mismo objeto
en memoria

        Console.WriteLine($"¿Es la misma instancia?
{ReferenceEquals(coordinador1, coordinador2)}");


        // Inicializa el pool de correos con 3 objetos precreados

        coordinador1.InicializarPool(3);

```

```

// Lista de destinatarios de ejemplo
var destinatarios = new List<string> {
    "cliente1@correo.com",
    "cliente2@correo.com",
    "cliente3@correo.com",
    "cliente4@correo.com",
    "cliente5@correo.com"

};

// Plantilla simple con marcador {nombre} para personalización
string plantilla = "Hola {sarahi}, este es un mensaje
personalizado.\nAtentamente, sarahicebsc@gmail.com";

// Llamada al método que envía correos masivos usando el pool y
coordinador singleton
coordinador1.EnviaCorreosMasivos(destinatarios, "Promoción especial",
plantilla);
}
}
}

```

Captura de ejecución

```
Consola de depuración de Mi x + v
Enviando correo a: cliente2@correo.com
Asunto: Promoción especial
Cuerpo: buenas tardes profesora maribel
Hola {sarahi}, este es un mensaje personalizado.
Atentamente, sarahicebsc@gmail.com
-----
Enviando correo a: cliente3@correo.com
Asunto: Promoción especial
Cuerpo: buenas tardes profesora maribel
Hola {sarahi}, este es un mensaje personalizado.
Atentamente, sarahicebsc@gmail.com
-----
Enviando correo a: cliente4@correo.com
Asunto: Promoción especial
Cuerpo: buenas tardes profesora maribel
Hola {sarahi}, este es un mensaje personalizado.
Atentamente, sarahicebsc@gmail.com
-----
Enviando correo a: cliente5@correo.com
Asunto: Promoción especial
Cuerpo: buenas tardes profesora maribel
Hola {sarahi}, este es un mensaje personalizado.
Atentamente, sarahicebsc@gmail.com
-----
C:\Users\l2021\OneDrive\Documents\Examen patrones de diseño unidad 2\Examen patrones de diseño unidad 2\bin\Debug\Examen
patrones de diseño unidad 2.exe (proceso 7172) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas -> Opciones -> Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana.
```

Conclusión

El uso combinado de los patrones singleton y object pool en este sistema permite crear una solución eficiente, escalable y muy bien estructurada para el envío masivo de correos electrónicos con plantillas. El patrón singleton es el que garantiza que exista una sola instancia global del coordinador de envíos, evitando conflictos y asegurando que todos los procesos de envío utilicen el mismo punto de control. Esto mejora la organización y sincronización del sistema, evitando la duplicación de tareas o conexiones redundantes. Mientras que por otra parte el patrón objeto pool es el que ayuda a la optimización y rendimiento al reutilizar objetos de correos preconfigurados y de esta manera reduciendo el tiempo de creación de nuevos objetos y consumo de recursos. En lugar de generar un nuevo objeto para cada correo, el sistema toma uno del pool, lo utiliza, y luego lo devuelve, permitiendo un flujo constante y eficiente. Al utilizar ambos patrones proporcionan una mejor arquitectura y mucho más sólida que reduce la carga del sistema, y mejora la velocidad de envío y no solo eso ayuda a mantener el control centralizado y ordenado sobre los correo masivos y las plantillas utilizadas, cumpliendo de manera óptima con los requisitos del problema que se presenta.